

FFN Design for Flow Matching

这些设计主要基于下面几条判断：

1. **Flow matching** 的监督目标本身是 **timestep-dependent** 的。

模型学习的是一个随时间变化的速度场

$$\mathcal{L}_{FM} = \mathbb{E}_{t, x_t \sim p_t} \|v_\theta(x_t, t) - u_t(x_t)\|^2$$

所以 FFN 若完全 time-agnostic，会天然和目标存在错位。

2. **Flow-based diffusion** 的训练目标具有明显的两阶段性质。

早期更偏向 global layout / mode navigation，后期更偏向 local detail refinement。也就是说，不同 t 下 FFN 需要承担的功能并不一样。

3. **JiT** 的结果说明，在高维生成问题里，“预测什么”比“网络是不是特别宽”更关键。

如果目标更接近低维、结构化的 clean signal，而不是高维无结构的 noisy quantity，那么带 bottleneck 的 Transformer block 也可能工作得很好。因此 FFN 设计不一定要追求“更大”，更关键的是让它更贴近 flow matching 的任务结构。

4. **LightningDiT** 的经验说明，**SwiGLU** 这类更强的 FFN 设计本身就会显著影响生成训练效率。

这说明 FFN 不是一个可忽略的配角，而是真正值得单独做 architecture search 的部分。

5. **ODE-ViT** 的连续深度观点提供了一个很自然的启发：

如果 Transformer block 可以被视作离散化的 dynamical update，那么 FFN 也可以从“状态更新器”而不是“静态 MLP”的角度来设计。

基于这些判断，下面给出六个独立方案。

1. Timestep-Adaptive Gated SwiGLU

核心想法

标准 SwiGLU 的形式可以写成：

$$\text{FFN}(x) = W_o \left(\text{SiLU}(W_g x) \odot W_v x \right).$$

它的问题不是表达力不够，而是 **gate** 和 **value** 都对 **timestep** 没有显式响应。

在 flow matching 里，不同 t 下模型应该激活不同类型的通道：早期更偏结构组织，后期更偏细节修正。因此最直接的改法不是重写整个 FFN，而是让 **gate 变成 timestep-aware**。

结构

令 t_{emb} 为时间嵌入，则将 gate 改成：

$$g(x, t) = \sigma(W_t t_{\text{emb}} + b_t)$$

或者更强一点，做成对每个 token 广播的门：

$$g(x, t) = \sigma(W_g x + U_t t_{\text{emb}})$$

最终输出写成：

$$y = W_o \left(\text{SiLU}(W_1 x) \odot W_2 x \odot g(x, t) \right)$$

这个设计为什么成立

这里的关键判断是：

timestep 更像是在决定“哪些中间通道该开、哪些该关”，而不是直接重写 token 内容本身。

也就是说，time signal 最适合进入 **gate**，因为 gate 本来就承担“信息通过/阻断”的角色。

这样改的好处是：

- value 路径仍然保持标准线性投影，信息保真；
- 改动非常轻，只新增一个从 t_{emb} 到中间维度的投影；
- 在 MoE 或 dense FFN 里都能直接用。

与已有工作的关系

- 和 AdaLN 类条件化相比，这里不是在归一化层上调 scale / shift，而是**直接调 FFN 内部非线性门控**，位置更深入。
- 和 LightningDiT 的 SwiGLU 改进方向一致：承认 FFN 内部结构本身会显著影响生成性能。
- 和两阶段 flow 现象相匹配：同一组通道在不同 t 下应该有不同重要性。

适合作为第一组实验的原因

这是六个方案里**最便宜、最稳**的一个。它不改 residual 形式，不改 token mixing，不引入 ODE 求解结构，只是在标准 FFN 内部加 time-aware gate。

如果这个方案都没有收益，说明“让 FFN 对 timestep 显式敏感”这个方向本身就需要重新评估。

2. Flow-Evolved Gate SwiGLU

核心理念

上一个方案只是“让 gate 接收 time signal”。这个方案再往前走一步：

不是只给 gate 一个额外 bias，而是让 **gate 在看到输入之前，先看到一个经过连续动力学演化后的输入**。

直觉上，gate 是在决定“当前该看什么特征”。

在 flow matching 中，早期和晚期需要 gate 看到的输入视角不一样：

- early t: 更适合看全局、粗结构、低频组织；
- late t: 更适合看局部、精细、可修正残差。

所以可以把 gate 输入做成一个随 t 连续变化的状态：

$$\tilde{x}(t) = \Phi_t(x)$$

其中 Φ_t 由一个很轻的低秩 ODE 近似得到。

结构

保留标准 SwiGLU 输出框架：

$$y = W_o \left(\text{SiLU}(W_1 \tilde{x}) \odot W_2 x \right)$$

这里只改 gate 路径，令

$$\tilde{x} = x + tAx + \frac{t^2}{2}A^2x + \dots$$

其中 A 用低秩分解表示：

$$A = UV^\top, \quad U, V \in \mathbb{R}^{d \times r}, \quad r \ll d$$

只取一阶时：

$$\tilde{x} = x + tUV^\top x$$

则 gate 的等效权重变成：

$$W_1^{\text{eff}}(t) = W_1(I + tUV^\top) = W_1 + t(W_1U)V^\top$$

也就是说，gate 的投影矩阵不再固定，而是在 timestep 上做一个 $\mathcal{O}(r)$ 的连续变形。

为什么只改 gate，不改 value

因为 gate 和 value 在 SwiGLU 里的职责并不对称：

- **gate** 决定该不该放行；
- **value** 提供真正传递的内容。

如果两边都做 ODE 演化，参数会明显变大，而且会把“控制路径”和“内容路径”同时扰动，训练更难稳。

只在 gate 输入上做连续演化，等于是在改“FFN 怎么看输入”，而不是改“FFN 传什么内容”。

这个设计最像什么

它和 ODE-ViT 的连续深度视角相通，但更局部：

ODE-ViT 是把 block 当 ODE；这里是把 **FFN 里的 gate 输入子空间** 当一个很轻的连续系统。

它又和 JiT 的 bottleneck 观察有潜在一致性：

既然生成任务里“结构化、低维有效变化”很重要，那么用低秩动力学去改 gate 的观察方向，本质上是在做一个更贴近任务的低维连续调制。

适合它的实验定位

这个方案的特点是：

- 比方案 1 更有“结构创新”；
- 参数增量仍可控；
- 明显带有论文感，因为它不只是加条件，而是把 gate 输入解释成 continuous deformation。

缺点是超参比方案 1 多，例如 γ 、阶数、时间缩放等。

3. Spatial-Adaptive Value SwiGLU

核心想法

flow-based diffusion 的两阶段现象说明：

- 早期更需要 global / coarse organization；
- 后期更需要 local / detail correction。

但标准 FFN 对每个 token 是逐点做 channel mixing，缺少一个很轻的“当前应偏向全局还是局部”的机制。

这个方案不做两路分支，而是只改 **value 输入**：把原始 token 和其局部上下文按 timestep 插值。

结构

先构造轻量空间上下文：

$$x_{\text{ctx}} = \text{SpatialMix}(x)$$

其中 SpatialMix 可以是：

- 2D reshape 后的 depthwise 3×3 conv；
- 若不能 reshape，则退化成 window mean / global mean 等轻量聚合。

然后根据 timestep 生成一个插值系数：

$$\alpha(t) = \sigma(W_{\alpha}t_{\text{emb}} + b_{\alpha})$$

将 value 输入改写为：

$$x_{\text{val}} = (1 - \alpha)x + \alpha x_{\text{ctx}}$$

最终 FFN 为：

$$y = W_o \left(\text{SiLU}(W_1 x) \odot W_2 x_{\text{val}} \right)$$

为什么 gate 看原始输入，value 看插值输入

这里的判断是：

- gate 决策需要高保真的局部信息，所以应该继续看原始 x ；
- value 是真正传递的内容，它可以根据 t 在“更原始的局部内容”和“更平滑的上下文内容”之间切换。

所以最自然的分工是：

$$\text{gate} \leftarrow x, \quad \text{value} \leftarrow x_{\text{val}}$$

这个设计和 Frequency-Split 的区别

它不是硬拆成两条 FFN 分支，而是保留一个完整容量的 SwiGLU，只在 value 输入前做一个 **time-controlled spatial interpolation**。

这比双分支方案更稳，原因是：

- 不用把 hidden dim 分成两半；
- 不会让每个分支都只拿到“半容量”；
- 更像是在一个强基线 FFN 上做小改，而不是重新造 block。

论文依据的逻辑来源

- 两阶段 flow 论文给了“早期偏结构、后期偏细节”的核心动机。
- LightningDiT 的结果说明 FFN 设计值得认真改，而不是只能靠 attention。
- 在视觉 backbone 中，depthwise conv 作为轻量局部混合手段已经非常成熟，因此把它放在 FFN value 前面是比较自然的工程落点。

这个方案的定位

如果你要找一个 **比方案 1 稍强、但比 ODE 方案更稳** 的设计，这个很合适。

它本质上是在 FFN 里加入一个“随时间调节的感受野偏置”。

4. Frequency-Split FFN

核心理想

如果进一步把“早期偏低频、后期偏高频”这个假设写死到结构里，那么最直接的做法就是显式做双分支：

- 一支负责 coarse / low-frequency；
- 一支负责 detail / high-frequency。

与方案 3 不同，这里不是一个 FFN + 插值输入，而是 **两个功能不同的 FFN 子模块**。

结构

先把输入拆成两种表示：

$$x_{\text{low}} = \text{LowPass}(x), \quad x_{\text{high}} = x - x_{\text{low}}$$

然后分别进入两个子 FFN：

$$h_{\text{low}} = f_{\text{low}}(x_{\text{low}}), \quad h_{\text{high}} = f_{\text{high}}(x_{\text{high}})$$

再由 timestep 生成混合比：

$$r(t) = \sigma(W_r t_{\text{emb}} + b_r)$$

最终输出：

$$y = r(t) h_{\text{low}} + (1 - r(t)) h_{\text{high}}$$

低频支路和高频支路应该怎么不同

如果两个支路只是输入不同、结构完全一样，那么其实 specialization 会比较弱。

所以更合理的做法是：

- **low branch**: 带局部聚合, 例如 depthwise conv, 或更平滑的通道参数化;
- **high branch**: 保留标准 pointwise SwiGLU, 更强调局部细节和尖锐变化。

例如:

$$h_{\text{low}} = W_o^{(l)} \left(\text{DWConv}(\text{SiLU}(W_1^{(l)} x_{\text{low}}) \odot W_2^{(l)} x_{\text{low}}) \right)$$

$$h_{\text{high}} = W_o^{(h)} \left(\text{SiLU}(W_1^{(h)} x_{\text{high}}) \odot W_2^{(h)} x_{\text{high}} \right)$$

这个设计的优点

它的 inductive bias 非常明确:

不是让模型自己“希望它学会两阶段”, 而是直接告诉它:

- 一部分容量用来处理 coarse;
- 一部分容量用来处理 detail;
- timestep 决定当前偏向哪一边。

这个设计的风险

它比方案 3 更“硬”, 所以风险也更明显:

1. hidden dim 被拆开以后, 单支表达力下降;
2. 如果 low / high 的分解方式不好, 可能把信息切坏;
3. 两支是否真的能学到频率 specialization, 不是自动保证的。

适合它的使用场景

如果你想要一个 **最容易讲 story、最像 paper figure 的结构**, 这个很好。

因为它能直观对应到 flow matching 的两阶段机制。

但如果你优先追求 first success, 我会把它排在方案 1 和方案 3 后面。

5. Progressive Refinement FFN

核心理念

flow matching 的另一个直觉不是“频率不同”, 而是 **同一个 block 内部也可以先做粗更新, 再做细修正**。

也就是说，一个 FFN 不一定非得是“一次算完”的静态非线性，它可以内部做成一个两步 refinement 过程。

这个想法和 ODE 视角也相容：

残差更新

$$z_{l+1} = z_l + f(z_l)$$

本来就可以被看成 Euler step。那我们完全可以在 **一个 FFN block 里面做一个小型 two-step update**。

结构

第一步先做标准粗变换：

$$h_1 = \text{SwiGLU}(x)$$

第二步再做一个更小的、time-aware refinement head：

$$\Delta h = g_\phi(h_1, t_{\text{emb}})$$

其中 g_ϕ 是一个小 MLP，例如隐藏维度只有主 FFN 的 1/4 或 1/8。

最后输出：

$$y = h_1 + \alpha(t) \Delta h$$

其中

$$\alpha(t) = \sigma(W_\alpha t_{\text{emb}} + b_\alpha)$$

为什么它是 FFN 设计，而不是单纯多加一层 MLP

关键点不在于“又加了一个小头”，而在于它把 FFN 的内部功能拆成了两个角色：

- **主干变换 h_1** ：承担绝大部分 token-wise 内容映射；
- **refinement head Δh** ：只负责 time-dependent 修正。

这样做等于在 block 内部显式编码了：

- 早期 t：更多依赖主干；
- 后期 t：让 refinement 更强。

和别的方案的差别

- 和方案 1 相比：这里不是调 gate，而是显式增加第二次修正；
- 和方案 4 相比：这里不是按频率拆两支，而是按“粗变换/细修正”拆两阶段；
- 和方案 6 相比：这里不是做多个共享 step 的积分，而是只做一个 very small refinement tail。

这个设计的优点

它特别适合 flow matching 这种“从粗到细”的任务叙事，因为结构本身就体现出先粗后细。

而且相比真正的多步积分，它仍然便宜很多。

风险

参数会比方案 1 和方案 3 更高一些。

如果 refinement head 做得太大，它可能会和主干功能重叠，最后变成“白加一层”。

6. Multi-Step Integrated FFN

核心想法

如果把 FFN 真正当作一个局部向量场更新器来看，那么单次 FFN 其实对应的是“一个大步长更新”。

但连续 flow 的真实形式更接近：

$$x(1) = x(0) + \int_0^1 v(x, t) dt$$

所以可以把一个 FFN block 内部改成 $\backslash(K\backslash)$ 个共享参数的小步更新，近似一个微型积分器。

结构

令 $x_0 = x$ ，然后在 block 内部做 K 步：

$$x_{k+1} = x_k + \frac{1}{K} f_\theta(x_k, t_k), \quad k = 0, \dots, K - 1$$

其中内部时间可以定义为：

$$t_k = t + \frac{k}{K} \Delta t$$

如果不想引入额外时间跨度，也可以简单设成：

$$t_k = t$$

只是在状态上递推。最终输出：

$$y = x_K$$

这里的 f_θ 可以是一个共享参数的轻量 SwiGLU / GLU 核心。

这个设计想解决什么

它解决的是一个“单步 FFN 太粗糙”的问题。

标准 FFN 相当于一次性把所有非线性都压进一个映射里，而多步结构更像在 token 表示空间里做几次小修正。

这对 flow matching 很自然，因为 flow 本来就是逐步 transport。

为什么它和普通堆层不一样

表面上看，好像只是“多做几次 MLP”。

但它和直接堆更多 Transformer layers 有两个本质差别：

1. 它发生在一个 **block** 内部，所以你是在改变 FFN 的局部归纳偏置；
2. **参数共享**，所以它更像数值积分，而不是增加新的独立层。

换句话说，它是在说：“一个 FFN block 不再代表一个静态函数，而代表一个小型求解器。”

这个设计和 ODE-ViT 的联系

ODE-ViT 把 ViT 的离散层解释为 ODE 离散化。这个方案把类似思想压缩到 FFN 子模块里：

不去重写整个 Transformer，而只把 FFN 这部分做成一个局部连续更新器。

风险

- 计算开销会近似乘上 K；
- 如果共享核太强，内部多步可能会迅速收缩到固定点，反而损失表达力；
- 需要控制稳定性，通常最好配合 residual scaling。

最后：六个方案的区别一句话总结

方案 1: Timestep-Adaptive Gated SwiGLU

不改 FFN 主体，只让 **gate** 对时间敏感。

方案 2: Flow-Evolved Gate SwiGLU

让 **gate** 看到一个经过低秩连续动力学演化后的输入。

方案 3: Spatial-Adaptive Value SwiGLU

让 **value** 在原始 token 与空间上下文之间随时间插值。

方案 4: Frequency-Split FFN

显式拆成 **coarse / detail** 两支 FFN，再由 timestep 决定混合。

方案 5: Progressive Refinement FFN

在一个 block 内部做 “主变换 + 小修正头” 两阶段更新。

方案 6: Multi-Step Integrated FFN

把一个 FFN block 变成 共享参数的小型积分器。

参考文献（建议在正式 proposal 里引用）

- **[R1]** Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, Matt Le. *Flow Matching for Generative Modeling*. 2023.
- **[R2]** Haoming Liu et al. *From Navigation to Refinement: Revealing the Two-Stage Nature of Flow-based Diffusion Models through Oracle Velocity*. 2025.
- **[R3]** Xingchao Liu, Chengyue Gong, Qiang Liu. *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*. 2022.
- **[R4]** Tianhong Li, Kaiming He. *Back to Basics: Let Denoising Generative Models Denoise (JiT)*. 2025.
- **[R5]** Jingfeng Yao, Bin Yang, Xinggang Wang. *Reconstruction vs. Generation: Taming Optimization Dilemma in Latent Diffusion Models / LightningDiT related results*. 2025.
- **[R6]** Carlos Boned Riera, David Romero Sánchez, Oriol Ramos Terrades. *ODE-ViT: Plug & Play Attention Layer from the Generalization of the ViT as an Ordinary Differential Equation*. 2025.